

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS

Escola Politécnica e de Artes — Análise e Desenvolvimento de Sistemas

ATIVIDADE PRÁTICA

Arquitetura Hexagonal com Java

Sistema de Gerenciamento de Clínica Veterinária

Disciplina	Análise e Desenvolvimento de Sistemas
Modalidade	Individual
Pré-requisito	Leitura do material sobre Arquitetura Hexagonal com Java
Entrega	Link do repositório Git público (GitHub/GitLab) via Moodle

1. Contexto e Cenário

Uma clínica veterinária de médio porte deseja modernizar o controle de seus atendimentos. Atualmente, os registros de consultas são feitos de forma manual e descentralizada, o que dificulta o acompanhamento do histórico clínico dos animais e gera inconsistências nos dados. A demanda é por um sistema Java que modele os principais casos de uso da clínica com organização arquitetural sólida e baixo acoplamento.

O sistema deve ser implementado utilizando exclusivamente a Arquitetura Hexagonal (Ports and Adapters). O código deve demonstrar, de forma verificável, que o domínio não possui dependência alguma da infraestrutura — o que será aferido pela ausência de imports de pacotes externos em classes do pacote de domínio.

Entidades do Domínio

O sistema é composto pelas seguintes entidades centrais:

- **Animal:** identificador, nome, espécie, raça, data de nascimento, nome do tutor.
 - Atributos: `Long id`, `String nome`, `String especie`, `String raca`, `LocalDate dataNascimento`, `String tutor`
- **Veterinario:** identificador, nome, CRMV, especialidade, situação (`DISPONIVEL / OCUPADO`).
- **Consulta:** identificador, animal, veterinário, data, hora, tipo (`ROTINA`, `EMERGENCIA`, `RETORNO`), situação (`AGENDADA`, `REALIZADA`, `CANCELADA`), observações.

2. Estrutura de Pacotes

O projeto deve ser organizado estritamente conforme a estrutura abaixo. A hierarquia de pacotes é parte essencial da demonstração arquitetural e não deve ser alterada.



Regra inviolável

Nenhuma classe do pacote `dominio` pode conter um `import` de classes dos pacotes `infraestrutura` ou `apresentacao`. O domínio deve compilar sem nenhuma dependência externa ao próprio pacote.

3. Etapa 1 — Modelagem do Domínio

3.1 Entidades e Enumerações

As três entidades do domínio devem ser implementadas como POJOs (Plain Old Java Objects), sem nenhuma anotação de framework externo (JPA, Spring, Jackson, etc.). As regras de negócio que envolvem o estado de cada entidade devem residir dentro da própria classe — não nos serviços.

3.1.1 Animal.java

- Implementar todos os atributos com visibilidade `private` (imutáveis com final quando aplicável).
- Implementar o método `int calcularIdadeEmAnos()`, que retorna a idade com base em `dataNascimento` e `LocalDate.now()`.
- O construtor deve lançar `IllegalArgumentException` se nome, espécie ou tutor forem nulos ou vazios.

3.1.2 Veterinario.java

- Implementar o método `boolean estaDisponivel()` com base no atributo `situacao`.
- O método `void ocupar()` altera a situação para `OCUPADO` apenas se o veterinário estiver `DISPONIVEL`; caso contrário, lança `VeterinarioIndisponivelException`.
- O método `void liberar()` reverte a situação para `DISPONIVEL` incondicionalmente.

3.1.3 Consulta.java

- Implementar as transições de estado por meio dos métodos `realizar(String observacoes)` e `cancelar()`.
- O método `realizar()` é permitido apenas quando a situação for `AGENDADA`.
- O método `cancelar()` é permitido nas situações `AGENDADA` e `REALIZADA`.
- Transições inválidas devem lançar `IllegalStateException` com mensagem descritiva.

```
// Padrão esperado para transições de estado em Consulta.java
public void realizar(String observacoes) {
    if (this.situacao != SituacaoConsulta.AGENDADA) {
        throw new IllegalStateException(
            "Apenas consultas AGENDADAS podem ser realizadas. " +
            "Situação atual: " + this.situacao);
    }
    this.situacao = SituacaoConsulta.REALIZADA;
    this.observacoes = observacoes;
}
```

3.2 Exceções de Domínio

- Criar `AnimalNaoEncontradoException` e `VeterinarioIndisponivelException` como subclasses de `RuntimeException`.
- Ambas devem receber uma mensagem descritiva no construtor e expô-la via `getMessage()`.

4. Etapa 2 — Definição das Portas

4.1 Portas de Saída

Cada porta de saída é uma interface Java definida no pacote `dominio/porta/saida`. Os métodos devem expressar intenções de negócio — evitar nomes que reflitam detalhes de persistência.

```
// PortaAnimalRepositorio.java
public interface PortaAnimalRepositorio {
```

```
void salvar(Animal animal);
Optional<Animal> buscarPorId(Long id);
List<Animal> listarPorTutor(String tutor);
List<Animal> listarTodos();
void remover(Long id);
}

// PortaVeterinarioRepositorio.java
public interface PortaVeterinarioRepositorio {
    void salvar(Veterinario vet);
    Optional<Veterinario> buscarPorId(Long id);
    List<Veterinario> buscarDisponiveis();
    List<Veterinario> buscarPorEspecialidade(String especialidade);
}

// PortaConsultaRepositorio.java
public interface PortaConsultaRepositorio {
    void salvar(Consulta consulta);
    Optional<Consulta> buscarPorId(Long id);
    List<Consulta> buscarPorAnimal(Long animalId);
    List<Consulta> buscarPorVeterinario(Long vetId);
    List<Consulta> listarAgendadas();
}

// PortaNotificacaoTutor.java
public interface PortaNotificacaoTutor {
    void notificarAgendamento(String tutor, Animal animal, Consulta consulta);
    void notificarCancelamento(String tutor, Animal animal, String motivo);
}
```

4.2 Porta de Entrada

A porta de entrada define os casos de uso que o domínio expõe ao mundo externo. O `ServicoAgendaConsulta` implementará esta interface.

```
// PortaAgendaConsulta.java
public interface PortaAgendaConsulta {

    // Caso de uso 1: agendar uma consulta
    Consulta agendarConsulta(Long animalId, Long veterinarioId,
                             LocalDate data, LocalTime hora,
                             TipoConsulta tipo);

    // Caso de uso 2: registrar a realização de uma consulta
    Consulta realizarConsulta(Long consultaId, String observacoes);

    // Caso de uso 3: cancelar uma consulta agendada
    void cancelarConsulta(Long consultaId);

    // Caso de uso 4: consultar o histórico de um animal
    List<Consulta> obterHistoricoAnimal(Long animalId);

    // Caso de uso 5: consultar a agenda de um veterinário
    List<Consulta> obterAgendaVeterinario(Long vetId);
}
```

5. Etapa 3 — Serviço de Domínio

5.1 ServicoAgendaConsulta.java

O serviço de domínio orquestra os casos de uso. Deve implementar `PortaAgendaConsulta` e depender exclusivamente das quatro portas de saída, recebidas via injeção por construtor. Nenhuma classe de infraestrutura pode ser importada ou instanciada diretamente no serviço.

```
// Assinatura esperada do construtor
public ServicoAgendaConsulta(
    PortaAnimalRepositorio      animalRepo,
    PortaVeterinarioRepositorio vetRepo,
    PortaConsultaRepositorio    consultaRepo,
    PortaNotificacaoTutor       notificacao) { ... }
```

5.1.1 agendarConsulta

- Buscar o `Animal` pelo ID; lançar `AnimalNaoEncontradoException` se não encontrado.
- Buscar o `Veterinario` pelo ID; lançar `VeterinarioIndisponivelException` se não disponível (verificar via `estaDisponivel()`).
- Invocar `veterinario.ocupar()` para reservar o profissional.
- Criar a `Consulta` com situação `AGENDADA`, persistir via `consultaRepo.salvar()` e notificar o tutor via `notificacao.notificarAgendamento()`.
- Retornar a consulta criada.

5.1.2 realizarConsulta

- Buscar a `Consulta`; lançar `RuntimeException` se não encontrada.
- Invocar `consulta.realizar(observacoes)` — a validação de estado é responsabilidade da entidade.
- Buscar o `Veterinario` associado e invocar `veterinario.liberar()`.
- Persistir as alterações e retornar a consulta atualizada.

5.1.3 cancelarConsulta

- Buscar a `Consulta` e invocar `consulta.cancelar()`.
- Se a consulta estiver associada a um veterinário, invocar `veterinario.liberar()`.
- Notificar o tutor via `notificacao.notificarCancelamento()`. Persistir o estado final.

5.1.4 obterHistoricoAnimal e obterAgendaVeterinario

- Delegar diretamente para `consultaRepo.buscarPorAnimal()` e `consultaRepo.buscarPorVeterinario()`, respectivamente.

6. Etapa 4 — Adaptadores de Infraestrutura

6.1 Repositórios em Memória

Implementar as três interfaces de repositório utilizando HashMap como estrutura de armazenamento interno. Cada classe deve residir no pacote infraestrutura/adaptador/persistencia e não pode ser referenciada por nenhuma classe do domínio.

```
// Esqueleto esperado para AnimalRepositorioMemoria.java
public class AnimalRepositorioMemoria implements PortaAnimalRepositorio {

    private final Map<Long, Animal> store = new HashMap<>();
    private long proximoId = 1L;

    @Override
    public void salvar(Animal animal) {
        // Atribuir ID automático se animal.getId() for nulo
        store.put(animal.getId(), animal);
    }

    @Override
    public Optional<Animal> buscarPorId(Long id) {
        return Optional.ofNullable(store.get(id));
    }

    @Override
    public List<Animal> listarPorTutor(String tutor) {
        return store.values().stream()
            .filter(a -> a.getTutor().equalsIgnoreCase(tutor))
            .collect(Collectors.toList());
    }

    // ... demais métodos
}
```

6.2 Adaptadores de Notificação

Implementar dois adaptadores para PortaNotificacaoTutor. A existência de dois adaptadores intercambiáveis é a principal demonstração prática do padrão Hexagonal nesta atividade.

6.2.1 NotificacaoConsole.java

- Exibir as notificações no terminal com formato estruturado e legível.
- Incluir obrigatoriamente: nome do tutor, nome do animal, data e hora da consulta, veterinário responsável e tipo.

6.2.2 NotificacaoCsv.java

- Registrar cada notificação como uma linha em um arquivo `notificacoes.csv`.
- Campos obrigatórios, separados por vírgula: timestamp, tipo_evento, tutor, animal, veterinário, data_consulta.
- Criar o cabeçalho do arquivo na primeira execução. Usar `FileWriter` com modo append para não sobrescrever registros anteriores.

```
// Saída esperada de NotificacaoConsole
// [AGENDAMENTO] Tutor: Ana Lima | Animal: Rex (Labrador)
//               Veterinário: Dr. Marcos (Clínica Geral)
```

```
//                               Data: 15/07/2025 às 14:30 | Tipo: ROTINA

// Linha esperada em notificacoes.csv
// 2025-07-10T09:15:00,AGENDAMENTO,Ana Lima,Rex,Dr. Marcos,2025-07-15T14:30
```

7. Etapa 5 — Composição e Demonstração

7.1 Main.java

A classe Main é o único ponto do sistema onde adaptadores concretos são instanciados e conectados às portas. Deve demonstrar explicitamente a troca de adaptador de notificação sem alterar o serviço de domínio.

```
// Composição com adaptador Console
PortaAnimalRepositorio    animais    = new AnimalRepositorioMemoria();
PortaVeterinarioRepositorio vets     = new VeterinarioRepositorioMemoria();
PortaConsultaRepositorio  consultas = new ConsultaRepositorioMemoria();
PortaNotificacaoTutor     notif     = new NotificacaoConsole();

PortaAgendaConsulta agenda = new ServicoAgendaConsulta(
    animais, vets, consultas, notif);

// Troca para CSV — zero linhas do domínio alteradas
PortaNotificacaoTutor notifCsv = new NotificacaoCsv("notificacoes.csv");
PortaAgendaConsulta agendaCsv = new ServicoAgendaConsulta(
    animais, vets, consultas, notifCsv);
```

7.2 Fluxo de Demonstração

O método main deve executar os passos abaixo na ordem indicada, exibindo a saída correspondente a cada operação:

1. Cadastrar dois animais com tutores distintos.
2. Cadastrar dois veterinários com especialidades diferentes.
3. Agendar uma consulta de ROTINA utilizando NotificacaoConsole.
4. Agendar uma consulta de EMERGENCIA utilizando NotificacaoCsv.
5. Realizar a primeira consulta, registrando observações clínicas.
6. Cancelar a segunda consulta e verificar a liberação do veterinário.
7. Exibir o histórico de consultas do primeiro animal.
8. Exibir a agenda do primeiro veterinário após os eventos anteriores.

Saída mínima esperada no console

```
[AGENDAMENTO] Tutor: João Silva | Animal: Thor | Vet.: Dra. Beatriz | Data: ...
[CONSULTA REALIZADA] Animal: Thor | Obs.: Exame de rotina sem alterações.
[CANCELAMENTO] Tutor: Maria Souza | Animal: Luna | Motivo: ...

=== Histórico de Thor ===
```

```
Consulta #1 - 15/07/2025 14:30 | ROTINA | REALIZADA
```

```
=== Agenda de Dra. Beatriz ===
```

```
Consulta #1 - 15/07/2025 14:30 | ROTINA | REALIZADA
```

8. Testes com Fakes (Opcional)

Esta etapa não é obrigatória, mas demonstra de forma conclusiva que o domínio é testável sem qualquer infraestrutura ativa. Os testes devem usar implementações simples das portas (fakes) em vez de frameworks de mock.

```
// FakeConsultaRepositorio.java - implementação de teste
public class FakeConsultaRepositorio implements PortaConsultaRepositorio {
    private final Map<Long, Consulta> store = new HashMap<>();

    @Override public void salvar(Consulta c) { store.put(c.getId(), c); }
    @Override public Optional<Consulta> buscarPorId(Long id) {
        return Optional.ofNullable(store.get(id));
    }
    // demais métodos retornam listas vazias ou Optional.empty()
}

// Cenário: veterinário indisponível deve lançar exceção
PortaAgendaConsulta agenda = new ServicoAgendaConsulta(
    new FakeAnimalRepositorio(),
    new FakeVeterinarioRepositorioOcupado(), // retorna vet OCUPADO
    new FakeConsultaRepositorio(),
    new FakeNotificacao());

try {
    agenda.agendarConsulta(1L, 1L, LocalDate.now(), LocalTime.of(10, 0),
        TipoConsulta.ROTINA);
    assert false : "Deveria ter lançado exceção.";
} catch (VeterinarioIndisponivelException e) {
    System.out.println("[PASSOU] " + e.getMessage());
}
```

Cenários sugeridos para implementação

- Agendamento bem-sucedido: verificar que a consulta retornada possui situação AGENDADA e que o veterinário foi marcado como OCUPADO.
- Veterinário indisponível: verificar que `VeterinarioIndisponivelException` é lançada ao tentar agendar com veterinário OCUPADO.
- Animal não encontrado: verificar que `AnimalNaoEncontradoException` é lançada ao usar um ID inexistente.
- Transição inválida: verificar que `IllegalStateException` é lançada ao tentar realizar uma consulta já CANCELADA.
- Cancelamento libera veterinário: verificar que `veterinario.estaDisponivel()` retorna true após o cancelamento da consulta.

9. Instruções de Entrega

O projeto deve ser entregue como repositório Git público (GitHub ou GitLab). O link deve ser submetido via Moodle até a data definida na plataforma.

9. Código-fonte organizado exatamente conforme a estrutura de pacotes da Seção 2.
10. Arquivo `README.md` contendo: instruções de compilação e execução; descrição das decisões arquiteturais; explicação de cada porta implementada e do adaptador correspondente.
11. Histórico de commits com mensagens descritivas, evidenciando desenvolvimento incremental (mínimo de 6 commits).
12. (Opcional) Classes de teste com fakes no pacote `test/`, executáveis com `java -ea`.

Referência para compilação e execução:

```
// Compilação e execução a partir do diretório src/  
find . -name "*.java" > sources.txt  
javac -d out @sources.txt  
java -cp out com.clinica.apresentacao.Main  
  
// Testes com assertions habilitadas (opcional):  
java -ea -cp out com.clinica.test.TestesDominio
```

Recomendações Técnicas

- **Java 11+** é suficiente. Java 17 é recomendado para suporte a records em objetos de valor.
- **Frameworks externos** (Spring, Quarkus, Hibernate, Jackson) não são permitidos. O objetivo é demonstrar os princípios com Java puro.
- Usar `Optional<T>` nos retornos de repositório para evitar retornos nulos e forçar o tratamento explícito de ausência de dados.
- Organizar commits de forma semântica: ex., `feat: implementa entidade Animal`, `feat: define PortaConsultaRepositorio`, `feat: adaptador NotificacaoCsv`.

Bom desenvolvimento.

A Arquitetura Hexagonal não é uma burocracia — é a garantia de que o domínio pode evoluir sem carregar o peso da infraestrutura.